

HW5 (20')

Submit as an HTML or PDF file

A note for PDF file:

It seems that we cannot directly save the Jupyter notebook as a PDF file due to a package loading error.

One way to achieve it is to:

Convert it to an HTML file -> Open the HTML file -> Right Click -> Print -> Save as PDF.

At least that is how I print out Quiz 2.

Print your name (First Name and Last Name) below.

```
In [1]: print ('Tej Shah')
```

Tej Shah

0. Make sure you sync forks and fetch origin the GitHub repository.

- In particular, you should see three folders `data`, `figures`, and `lecture notes` under your `XXX/GitHub/BI0S-584` folder.

1. Play with `os` library and basic commands (13')

- Import `numpy` and `pandas` library with proper aliases.
- Import a new library named `os` (1')
- This is the "operating system" library
 - Helpful for managing folders and special variables
 - No need to install
 - No need to rename

```
In [14]: # Write your own code
import numpy as np
import pandas as pd
import os
import importlib
```

- Use the new command `os.getcwd()` to show you the current working directory (CWD) in your computer
 - Usually refers to the folder where your Jupyter notebook is stored.
 - Assign `os.getcwd()` to a variable called `current_dir` and print it out. (1')
 - Note that `current_dir` is a string variable. You can use `type(current_dir)` function or `isinstance(current_dir, str)` to verify.

```
In [15]: # Write your own code
current_dir = os.getcwd()
print(type(current_dir))
print(current_dir)
```

```
<class 'str'>
/Users/Tej/Documents/GitHub/BIOS-584
```

- After getting the working directory, check the contents in the cwd.
- Use the command `os.listdir()` to show you a list of all relevant files under your CWD.
- Assign `os.listdir()` to a variable called `files_ls`. (1')
- Use a for loop to iterately show the variable name of the entire list. (1')
 - Each row will display one file name.
 - Each student will have different results.

```
In [16]: # Write your own code
files_ls = os.listdir()
for x in files_ls:
    print (x)
```

```

week-04-additional-list-and-for-loop.ipynb
.DS_Store
lecture notes
LICENSE
quiz1.html
week-05-application-simulation.ipynb
week-03-boolean-if-else.ipynb
week-02-jupyter.ipynb
quiz1_preparation.html
README.md
HW5.ipynb
HW1.ipynb
figures
.venv
quiz1_preparation.ipynb
HW3.ipynb
Quiz1.ipynb
week-02-variables-lists.ipynb
.ipynb_checkpoints
hw5_extension
HW4.ipynb
week-03-maths-arrays-random.ipynb
.git
data
test.ipynb
HW2.ipynb
.idea

```

- You should see a folder called `figures`. We will explore what it is.
- Change the working directory as the path to folder `figures` using the command `os.chdir()`.
 - Recall that the current working directory is stored using `os.getcwd()` and the output is a string.
- Use the format `"{}/figures".format(current_dir)` to create the new working directory. Assign the new string to `current_dir_2` (1')
- Verify your previous step using `os.getcwd()` and print it out. (1')
- Print out all files under the new directory (No need to use the for loop). (1')

```

In [17]: # Write your own code
current_dir_2 = "{}figures".format(current_dir)
os.chdir(current_dir_2)
print(os.getcwd())
print(os.listdir())

```

```

/Users/Tej/Documents/GitHub/BIOS-584/figures
['quiz1_prep_2.png', 'quiz1_prep_3.png', 'quiz1_prep_1.png', 'python_kernel.
png', 'quiz1_prep_4.png', 'project_flow.png', 'quiz1_q8.png', 'quiz1_q9.png',
'quiz1_prep_5.png', 'elif.png', 'forloop.png', 'grading_scale.png', 'pyc
harm_shortcut_package.png', 'if.png', 'ifelse.png', 'quiz1_q7.png', 'quiz1_q
6.png', 'quiz1_q4.png', 'quiz1_q5.png', 'quiz1_q1.png', 'quiz1_q2.png', 'qui
z1_q3.png']

```

- Change your working directory back to the original one and verify it (no need to show the files under original directory). (1')
 - For Madeline: Since each student has different directories, for this specific problem, check whether this directory is the same as the initial one. If yes, give them the full credit.

```
In [18]: # Write your own code
os.chdir(current_dir)
print(os.getcwd())
```

/Users/Tej/Documents/GitHub/BIOS-584

- After we change the working directory back to the original one, we learn how to create a new directory (or a new folder).
 - Create a new folder named `hw5_extension` under the current working directory.
 - Create a string following the format `"{}/hw5_extension".format(current_dir)` and use the new command `os.mkdir()`. (1')
 - Note that we do not need to assign `os.mkdir()` to a variable. `xxx = os.mkdir()` is wrong.
 - If this directory is already created, Python will yield an error. Make sure this is a new folder.

```
In [20]: # Write your own code
os.mkdir("{}hw5_extension".format(current_dir))
print(os.listdir())
```

```
['week-04-additional-list-and-for-loop.ipynb', '.DS_Store', 'lecture notes',
 'LICENSE', 'quiz1.html', 'week-05-application-simulation.ipynb', 'week-03-bo
olean-if-else.ipynb', 'week-02-jupyter.ipynb', 'quiz1_preparation.html', 'RE
ADME.md', 'HW5.ipynb', 'HW1.ipynb', 'figures', '.venv', 'quiz1_preparation.i
pynb', 'HW3.ipynb', 'Quiz1.ipynb', 'week-02-variables-lists.ipynb', '.ipynb_
checkpoints', 'hw5_extension', 'HW4.ipynb', 'week-03-maths-arrays-random.ipy
nb', '.git', 'data', 'test.ipynb', 'HW2.ipynb', '.idea']
```

Now it should be easier for you to load a dataset from any directory.

- Load the excel file `PTSD dataset.xlsx` using `pd.read_excel()` again and name it as `ptsd_df`. (1')
 - Note that this file is stored under the `data` folder.
 - Choose the sheet named `main_dataset`.
 - How should you spell out the directory? Use `'{}/data/PTSD data.xlsx'.format(current_dir)`
- Select 1st, 2nd, and 5th column names using `.iloc[:, ids_ls]`, where `ids_ls` is a list of relevant indices. (1')

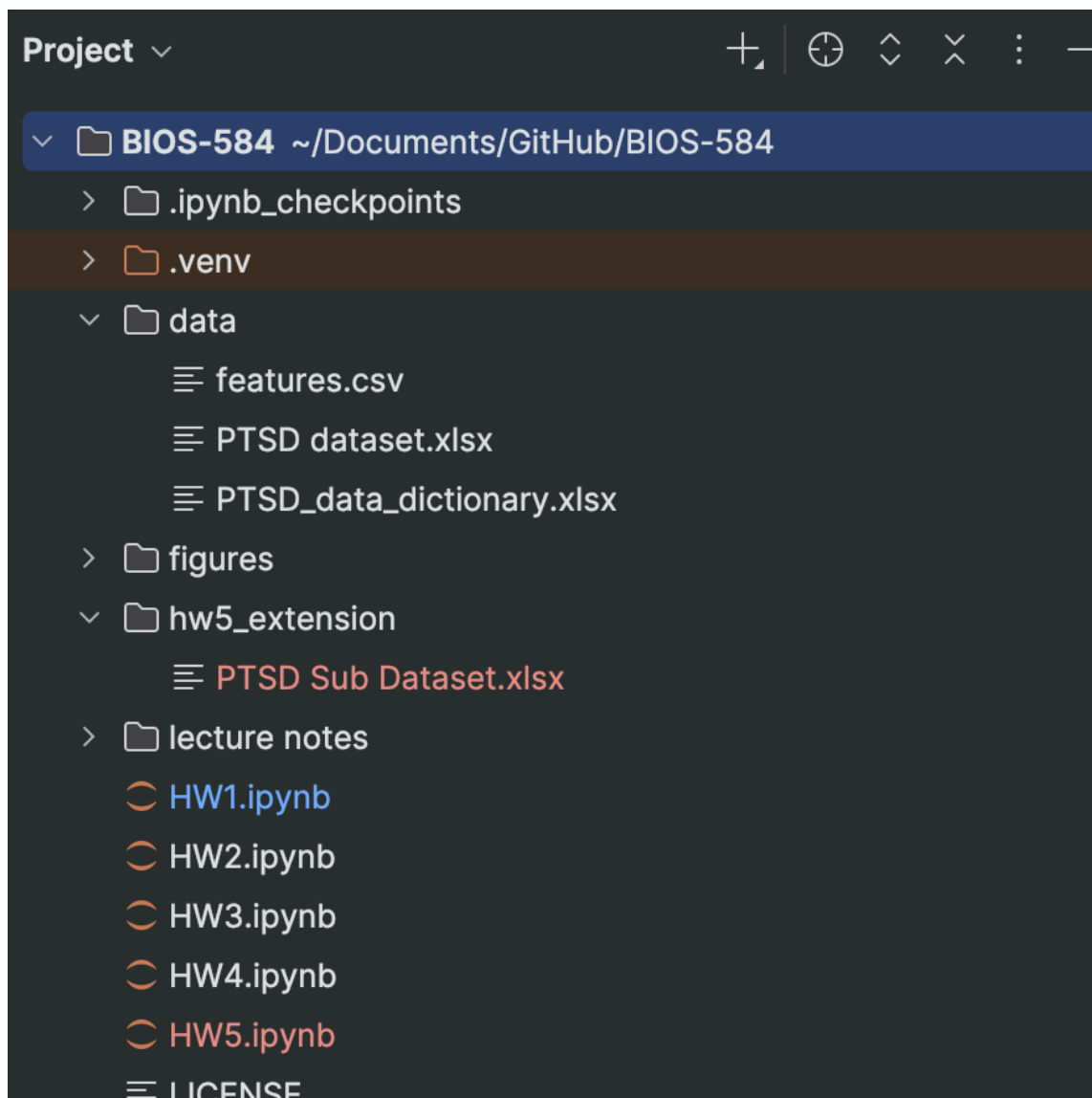
- Rename those three columns as a new dataset named `ptsd_sub_df`, save `ptsd_sub_df` under the previous `hw5_extension` folder using `pd.DataFrame.to_excel()` function with a sheet name `columns_1_2_5`, and give it a name as `PTSD Sub Dataset.xlsx`. (1')
 - Details can be found https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_excel.html.
 - There is a similar function for csv known as `read_csv()` and `to_csv()`.
- Provide a snapshot that includes your `hw5_extension` and the excel spreadsheet under it. You can save your snapshot under `hw5_extension` for plotting purposes. (1')

```
In [21]: # Write your own code
# Step 1: Load the PTSD dataset
ptsd_df = pd.read_excel('{}/data/PTSD dataset.xlsx'.format(current_dir), sheet_name='columns_1_2_5')

ids_ls = [0, 1, 4]
ptsd_sub_df = ptsd_df.iloc[:, ids_ls]
path = '{}/hw5_extension/PTSD Sub Dataset.xlsx'.format(current_dir)

pd.DataFrame.to_excel(ptsd_sub_df, path, sheet_name='columns_1_2_5', index = None)
```

Place your snapshot below (Make sure it is a Markdown)



2. Extension of for loop + if/else statements (7')

In the class, we illustrate simulating a $U \sim \text{Uniform}(0, 1)$ and set up a cutoff value to generate a binary variable with a certain probability distribution.

What if we have three treatment groups with a 2:2:1 ratio?

- Suppose we define our `N_total` as 200.
- Set a seed value with 100 (1')
- Initialize an empty list (or an array) to save the treatment group id.
- To simplify, let's call it 1, 2, and 3, and the proportion of treatment groups 1, 2, and 3 is approximately 2 : 2 : 1.
- Write a for loop that iterate `N_total` replications.
 - Within each iteration, generate a uniform variable `u_iter`. (1')
 - Properly define two cutoff values and apply `if/elif/else` statements to select the corresponding treatment groups (3')

- Convert `treatment_ls` to an array and assign it to `treatment_arr`.
- Print out the frequency and porportion of three treatment groups. (2')
 - The final output should look like
 - Group 1 has X patients (Y%). Group 2 has XX patients (YY%). Group 3 has XXX patients (YYY%).
 - You can use for loop to simplify, but it is optional.

```
In [13]: # write you own code
N_total = 200
np.random.seed(100)
treatment_ls = []

for n_iter in range(N_total):
    u_iter = np.random.uniform(low=0, high=1, size=None)
    if u_iter < 2/5:
        treatment_ls.append(1)
    elif 2/5 <= u_iter < 4/5:
        treatment_ls.append(2)
    else:
        treatment_ls.append(3)

treatment_arr = np.array(treatment_ls)
for group_id in [1, 2, 3]:
    count = np.sum(treatment_arr == group_id)
    proportion = count / N_total * 100
    print("Group {} has {} patients {:.1f}%".format(group_id, count, prop
```

Group 1 has 87 patients (43.5%).
 Group 2 has 72 patients (36.0%).
 Group 3 has 41 patients (20.5%).

In []: